Experiment No. 1

STUDY OF LOGIC GATES

AIM: To familiarize

i. TTL and CMOS Logic ICs for AND, OR, NOT, NAND, NOR and XOR by verification of truth tables.

COMPONENTS AND EQUIPMENTS REQUIRED:

SL No.	COMPONENTS	SPECIFICATION	QTY
1.	AND GATE	IC 7408	1
2.	OR GATE	IC 7432	1
3.	NOT GATE	IC 7404	1
4.	NAND GATE 2 I/P	IC 7400	1
5.	NOR GATE	IC 7402	1
6.	X-OR GATE	IC 7486	1
7.	IC TRAINER KIT	-	1
8.	Connecting Wires	As required	

PRINCIPLE:

Circuit that takes the logical decision and the process are called logic gates. Each gate has one or more input and only one output. OR, AND and NOT are basic gates

AND GATE:

The AND gate performs a logical multiplication commonly known as AND function. The output is high when both the inputs are high. The output is low level when any one of the inputs is low. Output Y= A.B where A and B are the inputs.

OR GATE:

The OR gate performs a logical addition commonly known as OR function. The output is high when any one of the inputs is high. The output is low level when both the inputs are low. Output Y= A+B where A and B are the inputs.

NOT GATE:

The NOT gate is called an inverter. The output is high when the input is low. The output is low when the input is high. Output Y= NOT(A) where A is the input.

NAND GATE:

The NAND gate is a combination of AND-NOT. The output is high when any one of the input is low or both inputs are low .The output is low level only when both inputs are high.

NOR GATE:

The NOR gate is a combination of OR-NOT. The output is high when both inputs are low. The output is low when one or both inputs are high.

X-OR GATE:

The output is high when any one of the inputs is high. The output is low when both the inputs are low and both the inputs are high.

PROCEDURE:

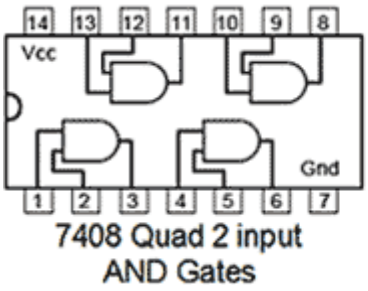
1. Check the all the required components.
2. Connect the circuit as per the circuit diagram.
3. Apply Vcc and Ground.
4. Logical inputs are given as per truth table using switches provided.
5. Observe the output on LEDs and verify the truth table.

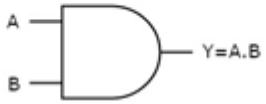
RESULT:

Familiarized with basic TTL and CMOS logic gate ICs and verified their truth tables.

AND GATE

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**

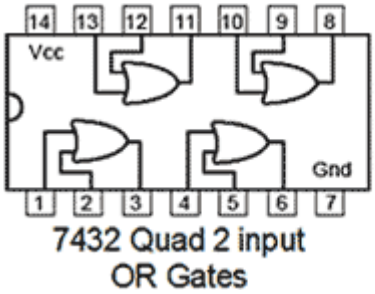


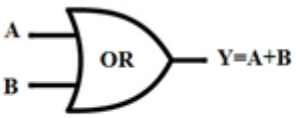


A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

OR GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**

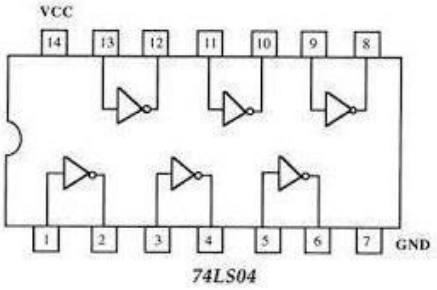


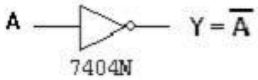


A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**

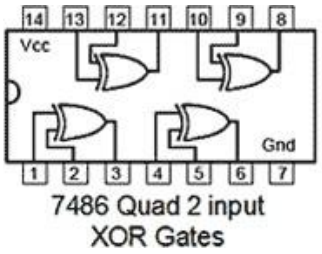




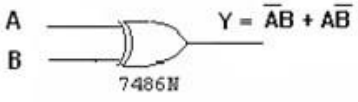
A	$\overline{A}$
0	1
1	0

X-OR GATE :

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**



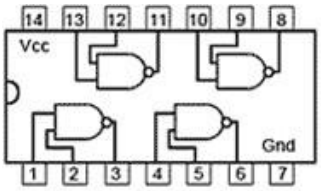
7486 Quad 2 input
XOR Gates



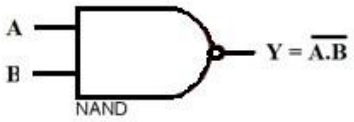
A	B	$\overline{A}B + A\overline{B}$
0	0	0
0	1	1
1	0	1
1	1	0

2-INPUT NAND GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**



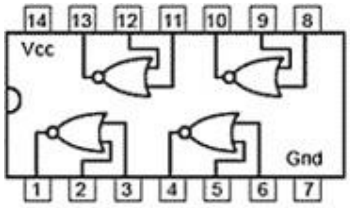
7400 Quad 2 input
NAND Gates



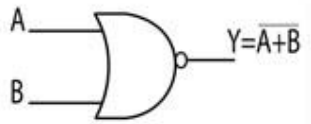
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE:

PIN DIAGRAM **SYMBOL** **TRUTH TABLE**



7402 Quad 2 input
NOR Gates



A	B	Y= $\overline{A+B}$
0	0	1
0	1	1
1	0	1
1	1	0

EXPERIMENT-2

De-Morgan's Theorem

AIM: Implement a logic circuit to verify demorgans theorem.

COMPONENTS REQUIRED:Trainer kit

THEORY:

De Morgan's Law Boolean Algebra defines the relation between the OR, AND, and the complements of variables, and is given for both the complement of AND and OR of two values. In Boolean Algebra there are two De Morgan's Laws that are:

- First De Morgan's Law
- Second De Morgan's Law

Let's understand these laws in detail as below:

First De Morgan's Law in Boolean Algebra

First De Morgan's law states that **"The complement of OR of two or more variables is equal to the AND of the complement of each variable."**
Let A and B be two variables, then mathematically First De Morgan's Law is given as:

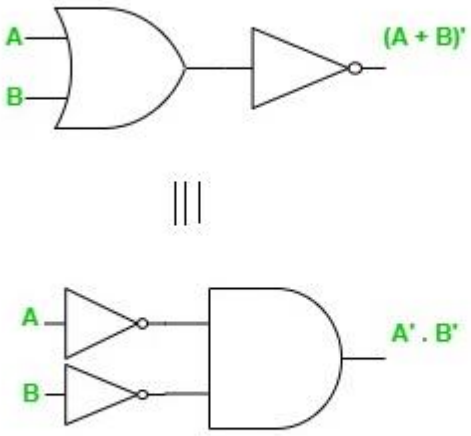
$$(A + B)' = A' . B'$$

Where

- *+ represents the OR operator between variables,*
- ***.** represents AND operator between variables, and*
- ***'** represents complement operation on variable.*

First De Morgan's Law Logic Gates

In context to logic gates and Boolean Algebra, De Morgan's Law states that "Both the logic gate circuits i.e., NOT gate is added to the output of OR gate, and NOT gate is added to the input of AND gate, are equivalent. These two logic gate circuits are given as follows:



First De Morgan's Law Truth Table

The truth table for first De Morgan's Law is given as follows:

A	B	A + B	(A + B)'	A'	B'	A'. B'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

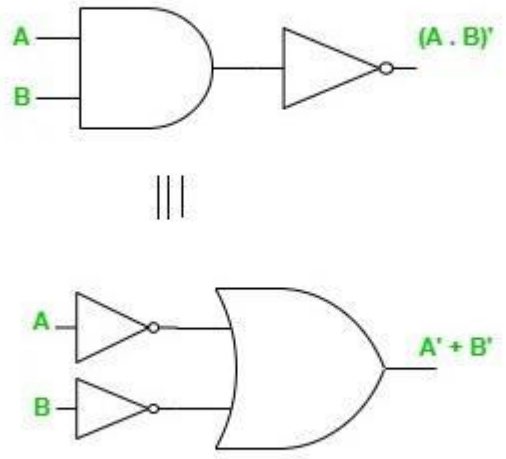
2 .Second De Morgan's law states that **"The complement of AND of two or more variables is equal to the OR of the complement of each variable."**
Let A and B be two variables, then mathematically Second De Morgan's Law is given as:

$$(A \cdot B)' = A' + B'$$

Second De Morgan's Law Truth Table

The truth table for the second De Morgan's Law is given as follows:

A	B	A . B	(A. B)'	A'	B'	A' + B'
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0



RESULT: Verified demorgans theorem successfully.

EXPERIMENT-3

BASIC GATES USING UNIVERSAL GATES

AIM: Implement basic gates using universal gates

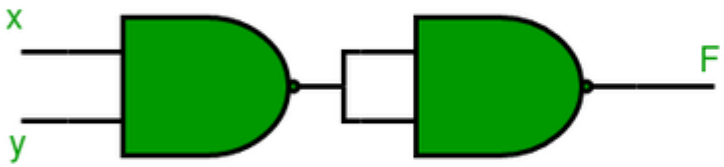
Components required : IC 7400,7402,Trainer kit

Theory: In Boolean Algebra, the **NAND** and **NOR** gates are called **universal gates** because any digital circuit can be implemented by using any one of these two i.e. any logic gate can be created using NAND or NOR gates only.

1. Implementation of AND Gate using Universal gates.

a) Using NAND Gates

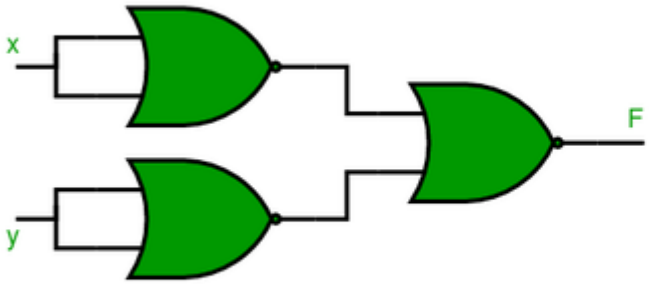
The AND gate can be implemented by using two NAND gates in the below fashion:



x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

b) Using NOR Gates

Implementation of AND gate using only NOR gates as shown below:

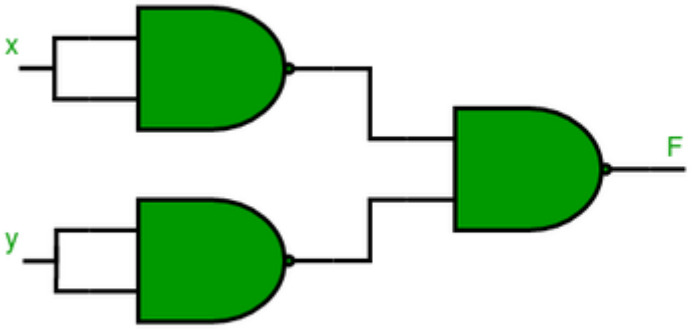


x	y	F
0	0	0
1	0	0
1	1	1
0	1	0

Implementation of OR Gate using Universal gates.

a) Using NAND Gates

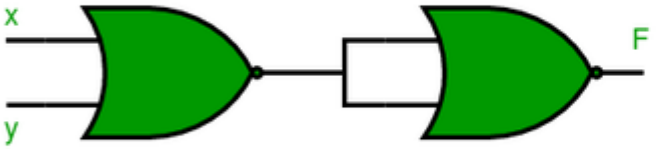
The OR gate can be implemented using the NAND gate as below:



x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

b) Using NOR Gates

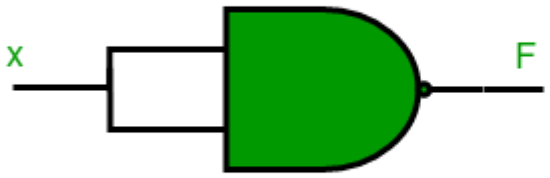
Implementation of OR gate using two NOR gates as shown in the picture below:



x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

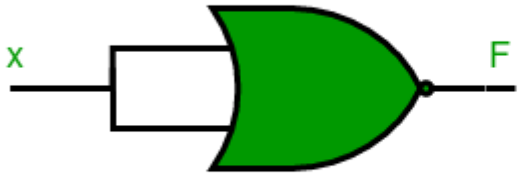
3. Implementation of NOT Gate using Universal gates.

a) Using NAND Gates



x	F
0	1
1	0

b) Using NOR Gates



x	F
0	1
1	0

EXPERIMENT-4

HALF ADDER AND HALF SUBSTRACTOR-BASIC GATES

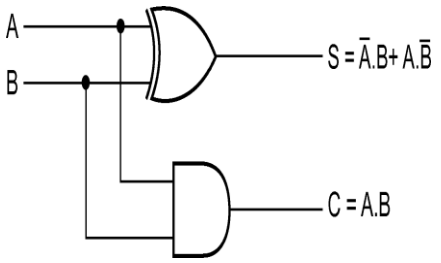
Aim 1: - To construct a Half adder circuit using basic gates

EQUIPMENTS / COMPONENTS REQUIRED :- Digital IC Trainer Kit, Ics – 7486, 7408

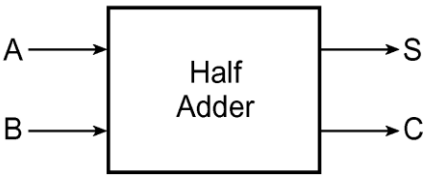
Theory:-
A half-adder is an arithmetic circuit block that can be used to add two bits. Such a circuit thus has two inputs that represent the two bits to be added and two outputs, with one producing the SUM output and the other producing the CARRY.
The Boolean expressions for the SUM and CARRY outputs are given by the equations

SUM S = A B + A B
CARRY C = A B

Procedure



A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



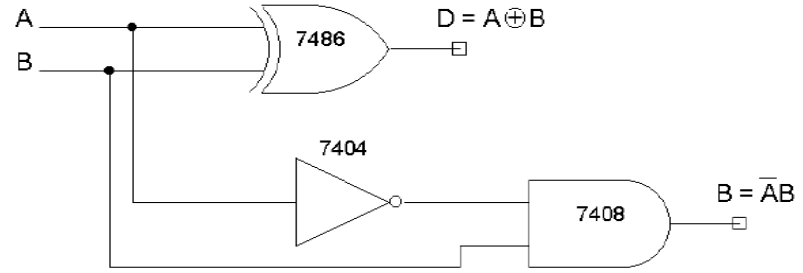
Aim 2: - To construct a Half subtracor circuit using basic gates

EQUIPMENTS / COMPONENTS REQUIRED :- Digital IC Trainer Kit, Ics – 7486, 7408,7404

Theory:-

The half subtractor is constructed using X-OR and AND Gate. The half subtractor has two input and two outputs. The outputs are difference and borrow. The difference can be applied using X-OR Gate, borrow output can be implemented using an AND Gate and an inverter.

Using X – OR and Basic Gates (a)Half Subtractor



INPUT		OUTPUT	
A	**B**	**BO**	**DIFF**
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

RESULT: - Study and verified truth-tables of Half adder and half substractor.

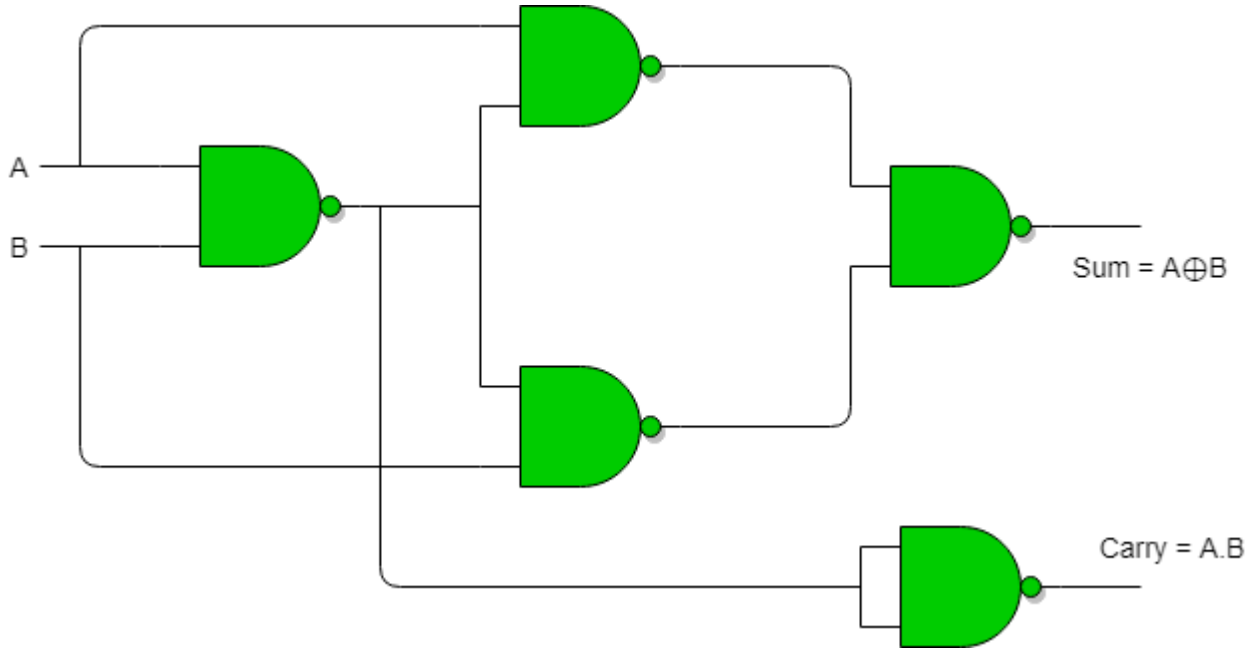
EXPERIMENT-5

HALF ADDER AND HALF SUBSTRACTOR-UNIVERSAL GATES

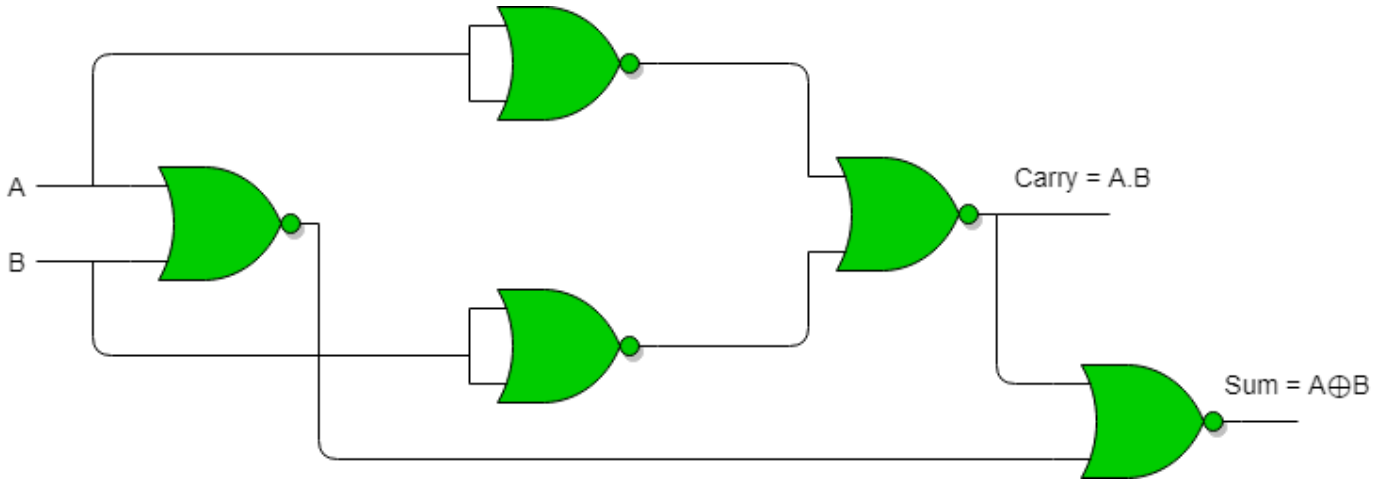
AIM: Implement half adder and half substractor using NAND and NOR gates

COMPONENTS REQUIRED: IC 7400,7402

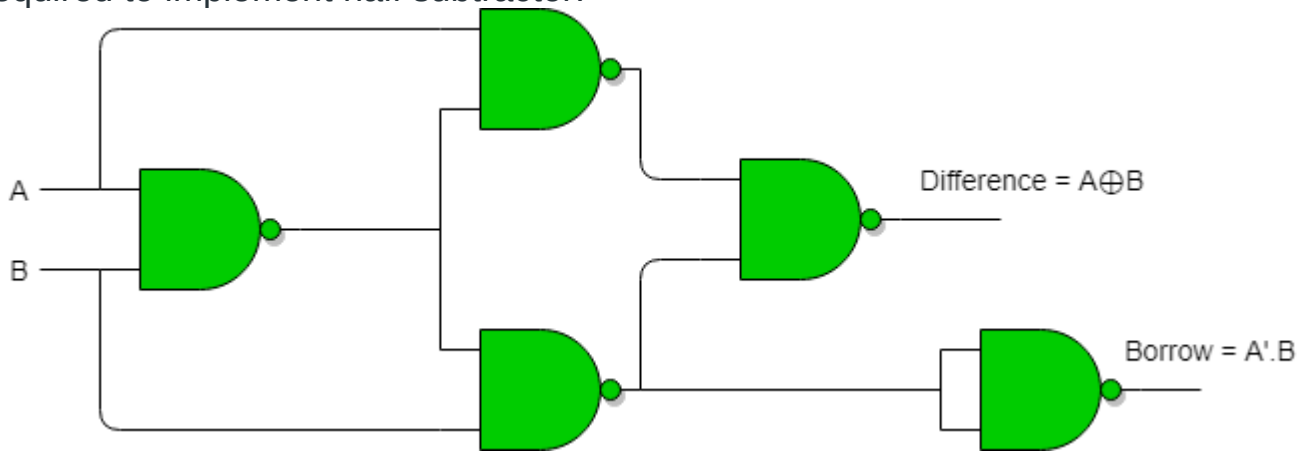
THEORY: Universality: NAND and NOR gates are considered universal gates because they can be used to implement any logical function, including binary arithmetic functions such as addition and subtraction.
Implementation of Half Adder using NAND gates : Total 5 NAND gates are required to implement half adder.



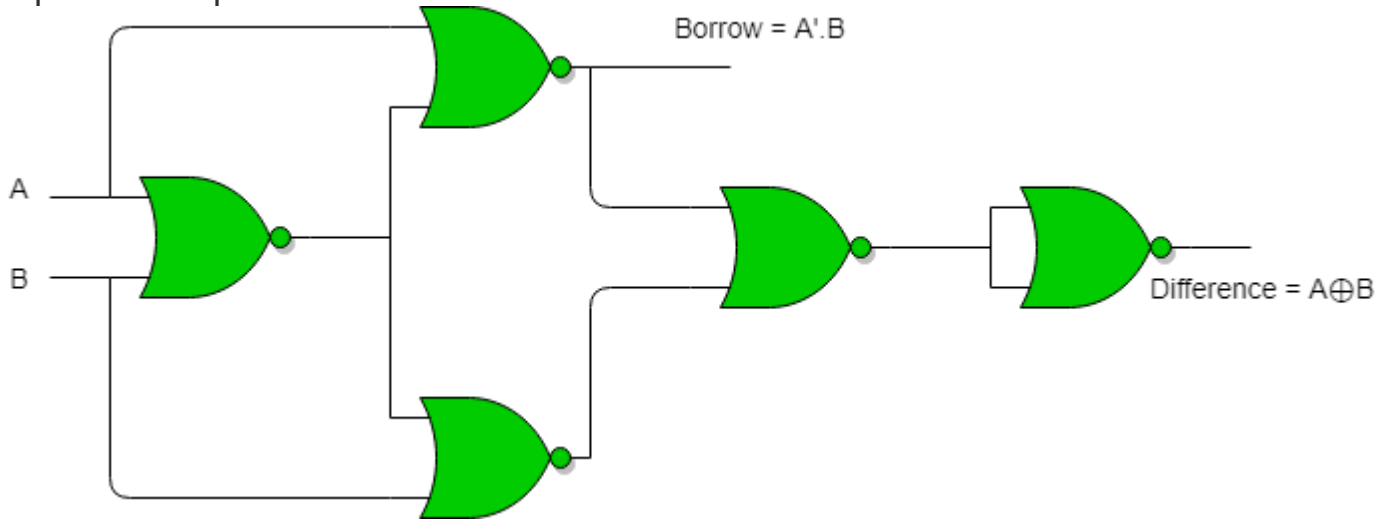
Implementation of Half Adder using NOR gates : Total 5 NOR gates are required to implement half adder.



Implementation of Half Subtractor using NAND gates : Total 5 NAND gates are required to implement half subtractor.



Implementation of Half Subtractor using NOR gates : Total 5 NOR gates are required to implement half subtractor.



EXPERIMENT-6

BINARY TO GRAY CONVERSION

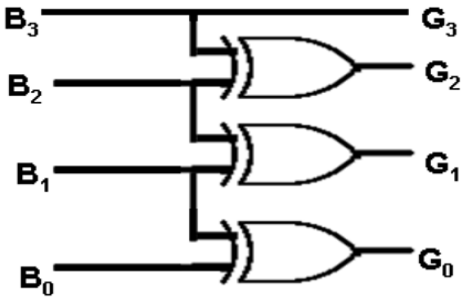
Aim: - To convert binary numbers to gray codes.

EQUIPMENTS / COMPONENTS REQUIRED :- Digital IC Trainer Kit, and IC 7486

Theory

A given binary number can be converted into its Gray code equivalent by going through the following steps:

1. Begin with the most significant bit (MSB) of the binary number. The MSB of the Gray code equivalent is the same as the MSB of the given binary number.
2. The second most significant bit, adjacent to the MSB, in the Gray code number is obtained by adding the MSB and the second MSB of the binary number and ignoring the carry, if any. That is, if the MSB and the bit adjacent to it are both '1', then the corresponding Gray code bit would be a '0'.
3. The third most significant bit, adjacent to the second MSB, in the Gray code number is obtained by adding the second MSB and the third MSB in the binary number and ignoring the carry, if any.
3. The process continues until we obtain the LSB of the Gray code number by the addition of the LSB and the next higher adjacent bit of the binary number.
4.
5. $B_3 = G_3$
6. $B_2 \oplus B_3 = G_2$
7. $B_1 \oplus B_2 = G_1$
8. $B_0 \oplus B_1 = G_0$



Inputs				Outputs			
B3	B2	B1	B0	G3 (V)	G2 (V)	G1 (V)	G0 (V)
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

GRAY TO BINARY CONVERSION

.

The Most Significant Bit (MSB) of the binary code is always equal to the MSB of the given binary number.

2.Other bits of the output binary code can be obtained by checking gray code bit at that index. If current gray code bit is 0, then copy previous binary code bit, else copy invert of previous binary code bit.

There are four inputs and four outputs. The input variable are defined as G_3, G_2, G_1, G_0 and the output variables are defined as B_3, B_2, B_1, B_0. From the truth table, combinational circuit is designed.The logical expressions are defined as :

$$G_0 \oplus G_1 \oplus G_2 \oplus G_3 = B_0$$

$$G_1 \oplus G_2 \oplus G_3 = B_1$$

$$G_2 \oplus G_3 = B_2$$

$$G_3 = B3$$

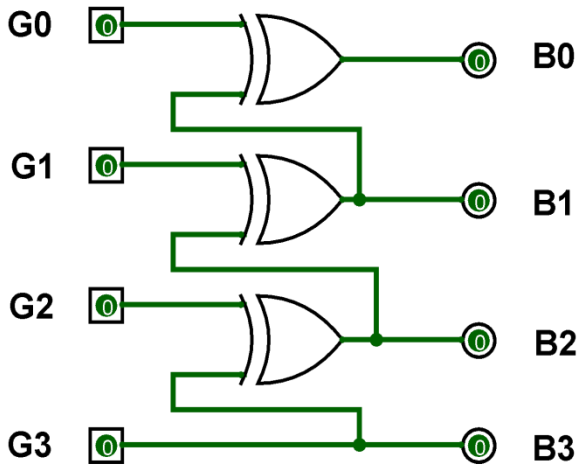


Figure-3: Gray to Binary Code Converter Circuit

Gray code				Natural-binary code			
G3	G2	G1	G0	B3	B2	B1	B0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
1	0	0	0	1	1	1	1
1	0	0	1	1	1	1	0
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	0	1	0	1	1
1	1	1	1	1	0	1	0

Figure-4: Gray to Binary Code Converter Truth Table